QUINTAURIS

RT-Europa Profile Release

2025-03-31

Table of Contents

Copyright and license information

Chapter 1. RISC-V Profiles

RISC-V was designed to provide a highly modular and extensible instruction set, and includes a large and growing set of standard extensions. In addition, users may add their own custom extensions. This flexibility can be used to highly optimize a specialized design by including only the exact set of ISA features required for an application, but the same flexibility also leads to a combinatorial explosion in possible ISA choices. Profiles specify a much smaller common set of ISA choices that capture the most value for most users, and which thereby enable the software community to focus resources on building a rich software ecosystem with application and operating system portability across different implementations.

Another pragmatic concern is the long and unwieldy ISA strings required to encode common sets of extensions, which will continue to grow as new extensions are defined.

Each profile is built on a standard base ISA plus a set of mandatory ISA extensions, and provides a small set of standard ISA options to extend the mandatory components. Profiles provide a convenient shorthand for describing the ISA portions of hardware and software platforms, and also guide the development of common software toolchains shared by different platforms that use the same profile. The intent is that the software ecosystem focus on supporting the profiles' mandatory base and standard options, instead of attempting to support every possible combination of individual extensions. Similarly, hardware vendors should aim to structure their offerings around standard profiles to increase the likelihood their designs will have mainstream software support.

Profiles are not intended to prohibit the use of combinations of individual ISA extensions or the addition of custom extensions, which can continue to be used for more specialized applications albeit without the expectation of widespread software support or portability between hardware platforms.

1.1. Profiles versus Platforms

Profiles only describe ISA features, not a complete execution environment.

A *software platform* is a specification for an execution environment, in which software targeted for that software platform can run.

A *hardware platform* is a specification for a hardware system (which can be viewed as a physical realization of an execution environment).

Both software and hardware platforms include specifications for many features beyond details of the ISA used by RISC-V harts in the platform (e.g., boot process, calling convention, behavior of environment calls, discovery mechanism, presence of certain memory-mapped hardware devices, etc.). Architecture profiles factor out ISA-specific definitions from platform definitions to allow ISA profiles to be reused across different platforms, and to be used by tools (e.g., compilers) that are common across many different platforms.

A platform can add additional constraints on top of those in a profile. For example, mandating an extension that is a standard option in the underlying profile, or constraining some implementation-specific parameter in the profile to lie within a certain range.

A platform cannot remove mandates or reduce other requirements in a profile.

Profiles are essential for ensuring long-term software portability across different RISC-V implementations. By defining a consistent ISA subset, profiles enable software developers to target a stable and well-documented execution environment, regardless of vendor-specific hardware variations. This guarantees that applications written for a given profile will function correctly across all conforming implementations. In contrast, platforms define execution environments that include system-level components such as firmware interfaces, memory-mapped devices, and boot protocols. While platforms may impose additional constraints on software, they cannot remove mandatory profile features or alter their defined behaviors.

1.2. Components of a Profile

1.2.1. Profile Family

Every profile is a member of a *profile family*. A profile family is a set of profiles that share the same base ISA but which vary in highest-supported privilege mode.

1.2.2. Profile Privilege Mode

RISC-V has a layered architecture supporting multiple privilege modes, and most RISC-V platforms support more than one privilege mode. Software is usually written assuming a particular privilege mode during execution. For example, application code is written assuming it will be run in user mode, and kernel code is written assuming it will be run in supervisor mode.

Software can be run in a mode different than the one for which it was written. For example, privileged code using privileged ISA features can be run in a user-mode execution environment, but will then cause traps into the enclosing execution environment when privileged instructions are executed. This behavior might be exploited, for example, to emulate a privileged execution environment using a user-mode execution environment.

The profile for a privilege mode describes the ISA features for an execution environment that has the eponymous privilege mode as the most-privileged mode available, but also includes all supported lower-privilege modes. In general, available instructions vary by privilege mode, and the behavior of RISC-V instructions can depend on the current privilege mode. For example, an S-mode profile includes U-mode as well as S-mode and describes the behavior of instructions when running in different modes in an S-mode execution environment, such as how an ecall instruction in U-mode causes a contained trap into an S-mode handler whereas an ecall in S-mode causes a requested trap out to the execution environment.

A profile may specify that certain conditions will cause a requested trap (such as an ecall made in the highest-supported privilege mode) or fatal trap to the enclosing execution environment. The profile does not specify the behavior of the enclosing execution environment in handling requested or fatal traps.

In particular, a profile does not specify the set of ECALLs available in the outer execution environment. This should be documented in the appropriate binary interface to the outer execution environment (e.g., Linux user ABI, or RISC-V SEE).

In general, a profile can be implemented by an execution environment using any hardware or software technique that provides compatible functionality, including pure software emulation.

A profile does not specify any invisible traps.

In particular, a profile does not constrain how invisible traps to a more-privileged mode can be used to emulate profile features.

A more-privileged profile can always support running software to implement a less-privileged profile from the same profile family. For example, a platform supporting the S-mode profile can run a supervisor-mode operating system that provides user-mode execution environments supporting the U-mode profile.

Instructions in a U-mode profile, which are all executed in user mode, have potentially different behaviors than instructions executed in user mode in an S-mode profile. For this reason, a U-mode profile cannot be considered a subset of an S-mode profile.

1.2.3. Profile ISA Features

An architecture profile has a mandatory ratified base instruction set (RV32I or RV64I for the current profiles). The profile also includes ratified ISA extensions placed into two categories:

1. Mandatory
2. Optional

As the name implies, *Mandatory ISA extensions* are a required part of the profile. Implementations of the profile must provide these. The combination of the profile base ISA plus the mandatory ISA extensions are termed the profile *mandates*, and software using the profile can assume these always exist.

The *Optional* category (also known as *options*) contains extensions that may be added as options, and which are expected to be generally supported as options by the software ecosystem for this profile.

The level of "support" for an Optional extension will likely vary greatly among different software components supporting a profile. Users would expect that software claiming compatibility with a profile would make use of any available supported options, but as a bare minimum software should not report errors or warnings when supported options are present in a system.

An optional extension may comprise many individually named and ratified extensions but a profile option requires all constituent extensions are present. In particular, unless explicitly listed as a profile option, individual extensions are not by themselves a profile option even when required as part of a profile option. For example, the Zbkb extension is not by itself a profile option even though it is a required component of the Zkn option.

Profile optional extensions are intended to capture the granularity at which the broad software ecosystem is expected to cope with combinations of extensions.

All components of a ratified profile must themselves have been ratified.

Platforms may provide a discovery mechanism to determine what optional extensions are present.

Extensions that are not explicitly listed in the mandatory or optional categories are termed *non-profile* extensions, and are not considered parts of the profile. Some non-profile extensions can be added to

an implementation without conflicting with the mandatory or optional components of a profile. In this case, the implementation is still compatible with the profile even though additional non-profile extensions are present. Other non-profile extensions added to an implementation might alter or conflict with the behavior of the mandatory or optional extensions in a profile, in which case the implementation would not be compatible with the profile.

Extensions that are released after a given profile is released are by definition non-profile extensions. For example, mandatory or optional profile extensions for a new profile might be prototyped as non-profile extensions on an earlier profile.

Chapter 2. QNT-RT Profile Class

The QNT-RT profile class targets realtime processors for markets requiring a deterministic behavior and rapid response times in applications.

2.1. QNT-RT Description

The Quintauris Real-Time RISC-V Profile Class (QNT-RT) defines a structured subset of the RISC-V ISA for a broad range of real-time processor classes. These profiles establish a core set of essential instructions that ensure deterministic execution, minimal latency, and predictable system behavior, meeting the demanding performance benchmarks of modern real-time systems.

2.2. QNT-RT Naming Scheme

The QNT-RT designation represents the real-time processor profile class from Quintauris. Each profile within this class is assigned an individual name according to an internal naming convention. This internal scheme helps categorize profiles based on ISA features, privilege levels, and real-time constraints, ensuring clarity for both hardware implementers and software developers.

This naming convention is specific to Quintauris and does not replace or conflict with RISC-V International (RVI) naming schemes. Instead, it serves as an internal structure to track profile variations and their evolving capabilities over time.

- Prefix **RT-** for Quintauris RISC-V Realtime
- Profile name
- Privilege mode:
  - **U** Unprivileged (available to any privilege mode, **U** is **not** User-mode)
  - **S** Supervisor mode (note that Hypervisor support is treated as an option)
  - **M** Machine mode
- A base ISA XLEN specifier (**32**, **64**)

Profile names are embeddable into RISC-V ISA naming strings. This implies that there will be no standard ISA extension with a name that matches the profile naming convention. This allows tools that process the RISC-V ISA naming string to parse and/or process a combined string. Quintauris does not impose the profile name to be directly adopted by RISC-V International.

2.3. QNT-RT Profile Releases

The following profile releases are defined in this profile class:

Name

RT-Europa

State

development

Release date

2025-03-31

Chapter 3. RT-Europa Profile Release

This profile release targets 32-bit realtimre processors for automotive.

RT-Europa has 90 associated implementation-defined parameters across all its defined profiles.

3.1. RT-Europa Description

The RT-Europa Profile is a 32-bit real-time processor profile within the QNT-RT class, defining a standardized subset of the RISC-V ISA for user-mode execution. It inherits the core objectives of QNT-RT, focusing on deterministic execution, minimal latency, and predictable system behavior. RT-Europa establishes a constrained set of mandatory and optional ISA features, ensuring efficient instruction processing, low-overhead system interaction, and execution consistency across different real-time processors.

The profile provides a stable execution model, defining execution constraints, required extensions, and implementation-dependent parameters to enable reliable software portability while maintaining compatibility with the broader RISC-V ecosystem.

3.2. RT-EuropaU32 Profile

The RT-EuropaU32 Profile defines the unprivileged execution environment for real-time processors, focusing on user-space software and task execution. It specifies the required ISA features, optional extensions, and implementation-defined parameters that shape how user-mode applications interact with the system. These definitions ensure a standardized execution model, enabling software portability and consistent behavior across different hardware implementations.

3.2.1. Mandatory Extensions

The RT-EuropaU32 Profile has 18 mandatory extensions.

- **U** User-mode privilege level

  Version = 1.0.0

- **I** Base integer ISA (RV32I or RV64I)

  Version = 2.1.0

- **A** Atomic instructions

  Version = 2.1.0

- **B** Bitmanipulation instructions

  Version = 1.0.0

- **M** Integer multiply and divide instructions

  Version = 2.0.0

- **Zicbom** Cache block management instructions

Version = 1.0.0

- **Zicbop** Cache block prefetch

  Version = 1.0.0

- **Zicboz** Cache block zero instruction

  Version = 1.0.0

- **Zicsr** Control and status registers

  Version = 2.0.0

- **Zce** Compressed instructions for microcontrollers

  Version = 1.0.0

  

  Specifying Zce on RV32 with F includes Zca, Zcb, Zcmp, Zcmt and Zcf

- **Zimop** May-be Operations

  Version ~> 1.0

  

  May-be operations

- **Zcmop** 16-bit May-be Operations

  Version = 1.0.0

  

  Commpressed (16-bit) may-be operations

- **Zihintpause** PAUSE instruction

  Version = 2.0.0

  

  While the `pause` *instruction is a HINT can be implemented as a NOP and hence trivially supported by hardware implementers, its inclusion in the mandatory extension list signifies that software should use the instruction whenever it would make sense and that implementers are expected to exploit this information to optimize hardware execution.*

- **Ziccamoa** Main memory atomicity requirement for RVA profiles

  Version = 1.0.0

- **Zicntr** Architectural performance counters

  Version = 2.0.0

- **Zihpm** Programmable hardware performance counters

  Version = 2.0.0

- **Zicond** Integer Conditional Operations

Version = 1.0.0

Integer conditional operations

- **Zihintntl** NTL (Non-Temporal Locality) Hint Instructions

  Version = 1.0.0

  Non-temporal memory-access locality hints

3.2.2. Optional Extensions

The RT-EuropaU32 Profile has 11 optional extensions.

- **Zilsd** Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **Zclsd** Compressed Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **F** Single-precision floating-point

  Version = 2.2.0

- **Zfh** Half-precision floating point

  Version ~> 1.0

  Mandatory if F extension is implemented.

- **Zcf** Compressed instructions for single precision floating point

  Version = 1.0.0

  Mandatory if F extension is implemented.

- **Zfa** Extension for Additional Floating-Point Instructions

  Version = 1.0.0

  Mandatory if F extension is implemented.

- **D** Double-precision floating-point

  Version = 2.2.0

- **V** Variable-length vector

  Version = 1.0.0

  The V extension is permitted with support targeting Zve64d. Implementations are not required to support VLEN = 128 bits, and a smaller VLEN is allowed as long as it meets the minimum architectural requirements of Zve64d.

- **Zfbfmin** Scalar BF16 Converts

  Version = 1.0.0

- **Zvfbfmin** Vector BF16 Converts

  Version = 1.0.0

  Mandatory if Zfbfmin and Zve64d extensions are implemented.

- **Zvfbfwma** Vector BF16 widening mul-add

  Version = 1.0.0

  Mandatory if Zfbfmin and Zve64d extensions are implemented.

3.2.3. Implementation-dependencies

RT-EuropaU32 has 24 associated implementation-defined parameters.

MUTABLE_MISA_U

Indicates whether or not the U extension can be disabled with the misa.U bit.

TRAP_ON_ECALL_FROM_U

Whether or not an ECALL-from-U-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

UXLEN

Set of XLENs supported in U-mode. Can be one of:

- 32: SXLEN is always 32
- 64: SXLEN is always 64
- 3264: SXLEN can be changed (via mstatus.UXL) between 32 and 64

U_MODE_ENDIANESS

Endianness of data in U-mode. Can be one of:

- little: U-mode data is always little endian
- big: U-mode data is always big endian
- dynamic: U-mode data can be either little or big endian, depending on the CSR field mstatus.UBE

LRSC_FAIL_ON_NON_EXACT_LRSC

Whether or not a Store Conditional fails if its physical address and size do not exactly match the physical address and size of the last Load Reserved in program order (independent of whether or not the SC is in the current reservation set)

LRSC_FAIL_ON_VA_SYNONYM

Whether or not an `sc.l`/sc.d will fail if its VA does not match the VA of the prior `lr.l`/lr.d, even if the

physical address of the SC and LR are the same

LRSC_MISALIGNED_BEHAVIOR

What to do when an LR/SC address is misaligned and MISALIGNED_AMO == false.

- 'always raise misaligned exception': self-explainitory
- 'always raise access fault': self-explainitory
- 'custom': Custom behavior; misaligned LR/SC may sometimes raise a misaligned exception and sometimes raise a access fault. Will lead to an 'unpredictable' call on any misaligned LR/SC access

LRSC_RESERVATION_STRATEGY

Strategy used to handle reservation sets.

- "reserve naturally-aligned 64-byte region": Always reserve the 64-byte block containing the LR/SC address
- "reserve naturally-aligned 128-byte region": Always reserve the 128-byte block containing the LR/SC address
- "reserve exactly enough to cover the access": Always reserve exactly the LR/SC access, and no more
- "custom": Custom behavior, leading to an 'unpredictable' call on any LR/SC

MISALIGNED_AMO

whether or not the implementation supports misaligned atomics in main memory

MUTABLE_MISA_A

When the A extensions is supported, indicates whether or not the extension can be disabled in the misa.A bit.

MUTABLE_MISA_B

Indicates whether or not the B extension can be disabled with the misa.B bit.

MUTABLE_MISA_M

Indicates whether or not the M extension can be disabled with the misa.M bit.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

FORCE_UPGRADE_CBO_INVAL_TO_FLUSH

When true, an implementation prohibits setting menvcfg.CBIE == 11 such that all cbo.inval instructions either trap (when menvcfg.CBIE == '00') or flush (when menvcfg.CBIE == '01').

When false, an implementation allows a true INVAL operation for cbo.inval, and thus supports the setting menvcfg.CBIE == 11.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

TIME_CSR_IMPLEMENTED

Whether or not a real hardware time CSR exists. Implementations can either provide a real CSR or emulate access at M-mode.

Possible values:

true

time/timeh exists, and accessing it will not cause an IllegalInstruction trap

false

time/timeh does not exist. Accessing the CSR will cause an IllegalInstruction trap or enter an unpredictable state, depending on TRAP_ON_UNIMPLEMENTED_CSR. Privileged software may emulate the time CSR, or may pass the exception to a lower level.

HW_MSTATUS_FS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.FS

Values are:

never	Hardware never writes mstatus.FS
precise	Hardware writes mstatus.FS to the Dirty (3) state precisely when F registers are modified
imprecise	Hardware writes mstatus.FS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write FP state.

MSTATUS_FS_LEGAL_VALUES

The set of values that mstatus.FS will accept from a software write.

MUTABLE_MISA_F

Indicates whether or not the F extension can be disabled with the misa.F bit.

MUTABLE_MISA_D

Indicates whether or not the D extension can be disabled with the misa.D bit.

HW_MSTATUS_VS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.VS

Values are:

never	Hardware never writes mstatus.VS
precise	Hardware writes mstatus.VS to the Dirty (3) state precisely when V registers are modified
imprecise	Hardware writes mstatus.VS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write vector state.

MSTATUS_VS_LEGAL_VALUES

The set of values that mstatus.VS will accept from a software write.

MUTABLE_MISA_V

Indicates whether or not the V extension can be disabled with the misa.V bit.

3.3. RT-EuropaS32 Profile

The RT-EuropaS32 Profile extends RT-EuropaU32 by defining supervisor-mode execution constraints. This enables support for real-time operating systems (RTOS) and system-level services while maintaining the deterministic characteristics of the profile.

The RT-EuropaS32 profile specifies the ISA features available to a supervisor-mode execution environment in 32-bit realtime processors. RT-EuropaS32 is based on privileged architecture version 1.13.

3.3.1. Mandatory Extensions

The RT-EuropaS32 Profile has 25 mandatory extensions.

- **S** Supervisor mode

  Version ~> 1.13

  Supervisor version 1.13

- **Zifencei** Instruction fence

  Version ~> 1.0

  Instruction-Fetch fence instruction.

- **Ssaia** Advanced Interrupt Architecture, S-mode extension

  Version = 1.0.0

  Its either AIA or CLIC

- **Ssclic** Core-Local Interrupt Controller, S-mode extension

  Version = 0.9.0

  Its either AIA or CLIC

- **Ssstrict** Unimplemented reserved encodings trap and no no-conforming extensions

  Version = 1.0.0

- **Sstc** Supervisor-mode timer interrupts

  Version = 0.9.0

- **Sstvecd** Direct exception vectoring

Version = 1.0.0

- **U** User-mode privilege level

  Version = 1.0.0

- **I** Base integer ISA (RV32I or RV64I)

  Version = 2.1.0

- **A** Atomic instructions

  Version = 2.1.0

- **B** Bitmanipulation instructions

  Version = 1.0.0

- **M** Integer multiply and divide instructions

  Version = 2.0.0

- **Zicbom** Cache block management instructions

  Version = 1.0.0

- **Zicbop** Cache block prefetch

  Version = 1.0.0

- **Zicboz** Cache block zero instruction

  Version = 1.0.0

- **Zicsr** Control and status registers

  Version = 2.0.0

- **Zce** Compressed instructions for microcontrollers

  Version = 1.0.0

  Specifying Zce on RV32 with F includes Zca, Zcb, Zcmp, Zcmt and Zcf

- **Zimop** May-be Operations

  Version ~> 1.0

  May-be operations

- **Zcmop** 16-bit May-be Operations

  Version = 1.0.0

  Commpressed (16-bit) may-be operations

- **Zihintpause** PAUSE instruction

  Version = 2.0.0

  

  While the `pause` *instruction is a HINT can be implemented as a NOP and hence trivially supported by hardware implementers, its inclusion in the mandatory extension list signifies that software should use the instruction whenever it would make sense and that implementers are expected to exploit this information to optimize hardware execution.*

- **Ziccamoa** Main memory atomicity requirement for RVA profiles

  Version = 1.0.0

- **Zicntr** Architectural performance counters

  Version = 2.0.0

- **Zihpm** Programmable hardware performance counters

  Version = 2.0.0

- **Zicond** Integer Conditional Operations

  Version = 1.0.0

  

  Integer conditional operations

- **Zihintntl** NTL (Non-Temporal Locality) Hint Instructions

  Version = 1.0.0

  

  Non-temporal memory-access locality hints

3.3.2. Optional Extensions

The RT-EuropaS32 Profile has 17 optional extensions.

- **Sha** The augmented hypervisor extension

  Version = 1.0.0

  

  Augmented Hypervisor mode, implies all these extensions to be present - H - Ssstateen - Shcounterenw - Shvstvala - Shtvala - Shvstvecd - Shvsatpa - Shgatpa

- **Svbare** Bare virtual addressing

  Version ~> 1.0

  

  Must be impmeneted if Sha extension is implemented The **satp** *mode Bare must be supported*

- **Sspmp** Physical Memory Protection for supervisor mode

  Version = 0.9.2

Must be impmeneted if Sha extension is implemented

- **Sscofpmf** Counter Overflow and Privilege Mode Filtering

  Version = 1.0.0

  Counter Overflow and Privilege Mode Filtering

- **Sscounterenw** Supervisor counter enable

  Version = 1.0.0

  For any **hpmcounter** *that is not read-only zero, the corresponding bit in* **scounteren** *must be writable.*

- **Ssccfg** Supervisor-mode counter configuration

  Version = 1.0.0

- **Zilsd** Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **Zclsd** Compressed Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **F** Single-precision floating-point

  Version = 2.2.0

- **Zfh** Half-precision floating point

  Version ~> 1.0

  Mandatory if F extension is implemented.

- **Zcf** Compressed instructions for single precision floating point

  Version = 1.0.0

  Mandatory if F extension is implemented.

- **Zfa** Extension for Additional Floating-Point Instructions

  Version = 1.0.0

  Mandatory if F extension is implemented.

- **D** Double-precision floating-point

  Version = 2.2.0

- **V** Variable-length vector

  Version = 1.0.0

The V extension is permitted with support targeting Zve64d. Implementations are not required to support VLEN = 128 bits, and a smaller VLEN is allowed as long as it meets the minimum architectural requirements of Zve64d.

- **Zfbfmin** Scalar BF16 Converts

  Version = 1.0.0

- **Zvfbfmin** Vector BF16 Converts

  Version = 1.0.0

  Mandatory if Zfbfmin and Zve64d extensions are implemented.

- **Zvfbfwma** Vector BF16 widening mul-add

  Version = 1.0.0

  Mandatory if Zfbfmin and Zve64d extensions are implemented.

3.3.3. Implementation-dependencies

RT-EuropaS32 has 55 associated implementation-defined parameters.

ASID_WIDTH
Number of implemented ASID bits. Maximum is 16 for XLEN==64, and 9 for XLEN==32

MSTATUS_FS_LEGAL_VALUES
The set of values that mstatus.FS will accept from a software write.

MSTATUS_FS_WRITEABLE
When S is enabled but F is not, mstatus.FS is optionally writeable.

This parameter only has an effect when both S and F mode are disabled.

MSTATUS_TVM_IMPLEMENTED
Whether or not mstatus.TVM is implemented.

When not implemented mstatus.TVM will be read-only-zero.

MSTATUS_VS_LEGAL_VALUES
The set of values that mstatus.VS will accept from a software write.

MSTATUS_VS_WRITEABLE
When S is enabled but V is not, mstatus.VS is optionally writeable.

This parameter only has an effect when both S and V mode are disabled.

MUTABLE_MISA_S
Indicates whether or not the S extension can be disabled with the misa.S bit.

REPORT_ENCODING_IN_STVAL_ON_ILLEGAL_INSTRUCTION

When true, stval is written with the encoding of an instruction that causes an `IllegalInstruction` exception.

When false stval is written with `0` when an `IllegalInstruction` exception occurs.

REPORT_VA_IN_MTVAL_ON_INSTRUCTION_PAGE_FAULT

When true, mtval is written with the virtual PC of an instructino when fetch causes an `InstructionPageFault`.

WHen false, mtval is written with `0` when an instruction fetch causes an `InstructionPageFault`.

REPORT_VA_IN_MTVAL_ON_LOAD_PAGE_FAULT

When true, mtval is written with the virtual address of a load when it causes a `LoadPageFault`.

WHen false, mtval is written with `0` when a load causes a `LoadPageFault`.

REPORT_VA_IN_MTVAL_ON_STORE_AMO_PAGE_FAULT

When true, mtval is written with the virtual address of a store when it causes a `StoreAmoPageFault`.

WHen false, mtval is written with `0` when a store causes a `StoreAmoPageFault`.

REPORT_VA_IN_STVAL_ON_BREAKPOINT

When true, stval is written with the virtual PC of the EBREAK instruction (same information as mepc).

When false, stval is written with `0` on an EBREAK instruction.

Regardless, stval is always written with a virtual PC when an external breakpoint is generated

REPORT_VA_IN_STVAL_ON_INSTRUCTION_ACCESS_FAULT

When true, stval is written with the virtual PC of an instructino when fetch causes an `InstructionAccessFault`.

WHen false, stval is written with `0` when an instruction fetch causes an `InstructionAccessFault`.

REPORT_VA_IN_STVAL_ON_INSTRUCTION_MISALIGNED

When true, stval is written with the virtual PC when an instruction fetch is misaligned.

When false, stval is written with `0` when an instruction fetch is misaligned.

Note that when IALIGN=16 (i.e., when the C or one of the `Zc*` extensions are implemented), it is impossible to generate a misaligned fetch, and so this parameter has no effect.

REPORT_VA_IN_STVAL_ON_INSTRUCTION_PAGE_FAULT

When true, stval is written with the virtual PC of an instructino when fetch causes an `InstructionPageFault`.

WHen false, stval is written with `0` when an instruction fetch causes an `InstructionPageFault`.

REPORT_VA_IN_STVAL_ON_LOAD_ACCESS_FAULT

When true, stval is written with the virtual address of a load when it causes a `LoadAccessFault`.

WHen false, stval is written with 0 when a load causes a `LoadAccessFault`.

REPORT_VA_IN_STVAL_ON_LOAD_MISALIGNED

When true, stval is written with the virtual address of a load instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, stval is written with 0 when a load address is misaligned and MISALIGNED_LDST is false.

REPORT_VA_IN_STVAL_ON_LOAD_PAGE_FAULT

When true, stval is written with the virtual address of a load when it causes a `LoadPageFault`.

WHen false, stval is written with 0 when a load causes a `LoadPageFault`.

REPORT_VA_IN_STVAL_ON_STORE_AMO_ACCESS_FAULT

When true, stval is written with the virtual address of a store when it causes a `StoreAmoAccessFault`.

WHen false, stval is written with 0 when a store causes a `StoreAmoAccessFault`.

REPORT_VA_IN_STVAL_ON_STORE_AMO_MISALIGNED

When true, stval is written with the virtual address of a store instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, stval is written with 0 when a store address is misaligned and MISALIGNED_LDST is false.

REPORT_VA_IN_STVAL_ON_STORE_AMO_PAGE_FAULT

When true, stval is written with the virtual address of a store when it causes a `StoreAmoPageFault`.

WHen false, stval is written with 0 when a store causes a `StoreAmoPageFault`.

SATP_MODE_BARE

Whether or not satp.MODE == Bare is supported.

SCOUNTENABLE_EN

Indicates which counters can delegated via scounteren

An unimplemented counter cannot be specified, i.e., if HPM_COUNTER_EN[3] is false, it would be illegal to set SCOUNTENABLE_EN[3] to true.

SCOUNTENABLE_EN[0:2] must all be false if Zicntr is not implemented. SCOUNTENABLE_EN[3:31] must all be false if Zihpm is not implemented.

STVAL_WIDTH

The number of implemented bits in stval.

Must be greater than or equal to *max*(`PHYS_ADDR_WIDTH`, `VA_SIZE`)

STVEC_MODE_DIRECT

Whether or not stvec.MODE supports Direct (0).

STVEC_MODE_VECTORED

Whether or not stvec.MODE supports Vectored (1).

SV_MODE_BARE

Whether or not writing mode=Bare is supported in the satp register.

SXLEN

Set of XLENs supported in S-mode. Can be one of:

- 32: SXLEN is always 32
- 64: SXLEN is always 64
- 3264: SXLEN can be changed (via mstatus.SXL) between 32 and 64

S_MODE_ENDIANESS

Endianness of data in S-mode. Can be one of:

- little: S-mode data is always little endian
- big: S-mode data is always big endian
- dynamic: S-mode data can be either little or big endian, depending on the CSR field mstatus.SBE

TRAP_ON_ECALL_FROM_S

Whether or not an ECALL-from-S-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY

For implementations that make satp.MODE read-only zero (always Bare, *i.e.*, no virtual translation is implemented), attempts to execute an SFENCE.VMA instruction might raise an illegal-instruction exception.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY indicates whether or not that exception occurs.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY has no effect when some virtual translation mode is supported.

MUTABLE_MISA_U

Indicates whether or not the U extension can be disabled with the misa.U bit.

TRAP_ON_ECALL_FROM_U

Whether or not an ECALL-from-U-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

UXLEN

Set of XLENs supported in U-mode. Can be one of:

- 32: SXLEN is always 32
- 64: SXLEN is always 64
- 3264: SXLEN can be changed (via mstatus.UXL) between 32 and 64

U_MODE_ENDIANESS

Endianness of data in U-mode. Can be one of:

- little: U-mode data is always little endian
- big: U-mode data is always big endian
- dynamic: U-mode data can be either little or big endian, depending on the CSR field mstatus.UBE

LRSC_FAIL_ON_NON_EXACT_LRSC

Whether or not a Store Conditional fails if its physical address and size do not exactly match the physical address and size of the last Load Reserved in program order (independent of whether or not the SC is in the current reservation set)

LRSC_FAIL_ON_VA_SYNONYM

Whether or not an `sc.l`/sc.d will fail if its VA does not match the VA of the prior `lr.l`/lr.d, even if the physical address of the SC and LR are the same

LRSC_MISALIGNED_BEHAVIOR

What to do when an LR/SC address is misaligned and MISALIGNED_AMO == false.

- 'always raise misaligned exception': self-explainitory
- 'always raise access fault': self-explainitory
- 'custom': Custom behavior; misaligned LR/SC may sometimes raise a misaligned exception and sometimes raise a access fault. Will lead to an 'unpredictable' call on any misaligned LR/SC access

LRSC_RESERVATION_STRATEGY

Strategy used to handle reservation sets.

- "reserve naturally-aligned 64-byte region": Always reserve the 64-byte block containing the LR/SC address
- "reserve naturally-aligned 128-byte region": Always reserve the 128-byte block containing the LR/SC address
- "reserve exactly enough to cover the access": Always reserve exactly the LR/SC access, and no more
- "custom": Custom behavior, leading to an 'unpredictable' call on any LR/SC

MISALIGNED_AMO

whether or not the implementation supports misaligned atomics in main memory

MUTABLE_MISA_A

When the A extensions is supported, indicates whether or not the extension can be disabled in the misa.A bit.

MUTABLE_MISA_B

Indicates whether or not the B extension can be disabled with the misa.B bit.

MUTABLE_MISA_M

Indicates whether or not the M extension can be disabled with the misa.M bit.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

FORCE_UPGRADE_CBO_INVAL_TO_FLUSH

When true, an implementation prohibits setting menvcfg.CBIE == 11 such that all cbo.inval instructions either trap (when menvcfg.CBIE == '00') or flush (when menvcfg.CBIE == '01').

When false, an implementation allows a true INVAL operation for cbo.inval, and thus supports the setting menvcfg.CBIE == 11.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

TIME_CSR_IMPLEMENTED

Whether or not a real hardware time CSR exists. Implementations can either provide a real CSR or emulate access at M-mode.

Possible values:

true

time/timeh exists, and accessing it will not cause an IllegalInstruction trap

false

time/timeh does not exist. Accessing the CSR will cause an IllegalInstruction trap or enter an unpredictable state, depending on TRAP_ON_UNIMPLEMENTED_CSR. Privileged software may emulate the time CSR, or may pass the exception to a lower level.

HW_MSTATUS_FS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.FS

Values are:

never	Hardware never writes mstatus.FS
precise	Hardware writes mstatus.FS to the Dirty (3) state precisely when F registers are modified
imprecise	Hardware writes mstatus.FS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write FP state.

MSTATUS_FS_LEGAL_VALUES

The set of values that mstatus.FS will accept from a software write.

MUTABLE_MISA_F

Indicates whether or not the F extension can be disabled with the misa.F bit.

MUTABLE_MISA_D

Indicates whether or not the D extension can be disabled with the misa.D bit.

HW_MSTATUS_VS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.VS

Values are:

never	Hardware never writes mstatus.VS
precise	Hardware writes mstatus.VS to the Dirty (3) state precisely when V registers are modified
imprecise	Hardware writes mstatus.VS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write vector state.

MSTATUS_VS_LEGAL_VALUES

The set of values that mstatus.VS will accept from a software write.

MUTABLE_MISA_V

Indicates whether or not the V extension can be disabled with the misa.V bit.

3.4. RT-EuropaM32 Profile

The RT-EuropaM32 Profile defines the machine-mode execution environment for real-time processors, ensuring a predictable and secure foundation for low-level system control, interrupt handling, and hardware management.

The RT-EuropaM32 profile specifies the ISA features available to a machine-mode execution environment in 32-bit realtime processors. RT-EuropaM32 is based on privileged architecture version 1.13.

3.4.1. Mandatory Extensions

The RT-EuropaM32 Profile has 31 mandatory extensions.

- **Sm** Machine mode

  Version ~> 1.13

- **Smaia** Advanced Interrupt Architecture, M-mode extension

  Version = 1.0.0

  > *Its either AIA or CLIC*

- **Smclic** Core-Local Interrupt Controller, M-mode extension

  Version = 0.9.0

Its either AIA or CLIC

- **Smpmp** Physical Memory Protection

  Version = 1.13.0

- **Smepmp** PMP Enhancements for Memory Access and Execution Prevention on Machine Mode

  Version = 1.0.0

- **Smrnmi** Resumable Non-Maskable Interrupts

  Version = 1.0.0

- **S** Supervisor mode

  Version ~> 1.13

  Supervisor version 1.13

- **Zifencei** Instruction fence

  Version ~> 1.0

  Instruction-Fetch fence instruction.

- **Ssaia** Advanced Interrupt Architecture, S-mode extension

  Version = 1.0.0

  Its either AIA or CLIC

- **Ssclic** Core-Local Interrupt Controller, S-mode extension

  Version = 0.9.0

  Its either AIA or CLIC

- **Ssstrict** Unimplemented reserved encodings trap and no no-conforming extensions

  Version = 1.0.0

- **Sstc** Supervisor-mode timer interrupts

  Version = 0.9.0

- **Sstvecd** Direct exception vectoring

  Version = 1.0.0

- **U** User-mode privilege level

  Version = 1.0.0

- **I** Base integer ISA (RV32I or RV64I)

Version = 2.1.0

- **A** Atomic instructions

  Version = 2.1.0

- **B** Bitmanipulation instructions

  Version = 1.0.0

- **M** Integer multiply and divide instructions

  Version = 2.0.0

- **Zicbom** Cache block management instructions

  Version = 1.0.0

- **Zicbop** Cache block prefetch

  Version = 1.0.0

- **Zicboz** Cache block zero instruction

  Version = 1.0.0

- **Zicsr** Control and status registers

  Version = 2.0.0

- **Zce** Compressed instructions for microcontrollers

  Version = 1.0.0

  

  Specifying Zce on RV32 with F includes Zca, Zcb, Zcmp, Zcmt and Zcf

- **Zimop** May-be Operations

  Version ~> 1.0

  

  May-be operations

- **Zcmop** 16-bit May-be Operations

  Version = 1.0.0

  

  Commpressed (16-bit) may-be operations

- **Zihintpause** PAUSE instruction

  Version = 2.0.0

  

  While the `pause` *instruction is a HINT can be implemented as a NOP and hence trivially supported by hardware implementers, its inclusion in the mandatory extension list signifies that software should use the instruction whenever it would make sense and that implementers are expected to exploit this information to*

optimize hardware execution.

- **Ziccamoa** Main memory atomicity requirement for RVA profiles

  Version = 1.0.0

- **Zicntr** Architectural performance counters

  Version = 2.0.0

- **Zihpm** Programmable hardware performance counters

  Version = 2.0.0

- **Zicond** Integer Conditional Operations

  Version = 1.0.0

  Integer conditional operations

- **Zihintntl** NTL (Non-Temporal Locality) Hint Instructions

  Version = 1.0.0

  Non-temporal memory-access locality hints

3.4.2. Optional Extensions

The RT-EuropaM32 Profile has 20 optional extensions.

- **Smdbltrp** Double trap

  Version = 1.0.0

- **Smcdeleg** Performance counter delegation

  Version = 1.0.0

- **Smcntrpmf** Cycle and Instret Privilege Mode Filtering

  Version = 1.0.0

- **Sha** The augmented hypervisor extension

  Version = 1.0.0

  Augmented Hypervisor mode, implies all these extensions to be present - H - Ssstateen - Shcounterenw - Shvstvala - Shtvala - Shvstvecd - Shvsatpa - Shgatpa

- **Svbare** Bare virtual addressing

  Version ~> 1.0

  Must be impmeneted if Sha extension is implemented The **satp** *mode Bare must be supported*

- **Sspmp** Physical Memory Protection for supervisor mode

  Version = 0.9.2

  > *Must be impmeneted if Sha extension is implemented*

- **Sscofpmf** Counter Overflow and Privilege Mode Filtering

  Version = 1.0.0

  > *Counter Overflow and Privilege Mode Filtering*

- **Sscounterenw** Supervisor counter enable

  Version = 1.0.0

  > *For any* **hpmcounter** *that is not read-only zero, the corresponding bit in* **scounteren** *must be writable.*

- **Ssccfg** Supervisor-mode counter configuration

  Version = 1.0.0

- **Zilsd** Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **Zclsd** Compressed Load/Store Pair Instructions for RV32

  Version = 1.0.0

- **F** Single-precision floating-point

  Version = 2.2.0

- **Zfh** Half-precision floating point

  Version ~> 1.0

  > *Mandatory if F extension is implemented.*

- **Zcf** Compressed instructions for single precision floating point

  Version = 1.0.0

  > *Mandatory if F extension is implemented.*

- **Zfa** Extension for Additional Floating-Point Instructions

  Version = 1.0.0

  > *Mandatory if F extension is implemented.*

- **D** Double-precision floating-point

  Version = 2.2.0

- **V** Variable-length vector

  Version = 1.0.0

  > *The V extension is permitted with support targeting Zve64d. Implementations are not required to support VLEN = 128 bits, and a smaller VLEN is allowed as long as it meets the minimum architectural requirements of Zve64d.*

- **Zfbfmin** Scalar BF16 Converts

  Version = 1.0.0

- **Zvfbfmin** Vector BF16 Converts

  Version = 1.0.0

  > *Mandatory if Zfbfmin and Zve64d extensions are implemented.*

- **Zvfbfwma** Vector BF16 widening mul-add

  Version = 1.0.0

  > *Mandatory if Zfbfmin and Zve64d extensions are implemented.*

3.4.3. Implementation-dependencies

RT-EuropaM32 has 90 associated implementation-defined parameters.

ARCH_ID
: Vendor-specific architecture ID in marchid

CONFIG_PTR_ADDRESS
: Physical address of the unified discovery configuration data structure. This address is reported in the mconfigptr CSR.

IMP_ID
: Vendor-specific implementation ID in mimpid

MISALIGNED_LDST
: Does the implementation perform non-atomic misaligned loads and stores to main memory (does **not** affect misaligned support to device memory)? If not, the implementation always throws a misaligned exception.

MISALIGNED_LDST_EXCEPTION_PRIORITY
: The relative priority of a load/store/AMO exception vs. load/store/AMO page-fault or access-fault exceptions.

  May be one of:

low	Misaligned load/store/AMO exceptions are always lower priority than load/store/AMO page-fault and access-fault exceptions.

high	Misaligned load/store/AMO exceptions are always higher priority than load/store/AMO page-fault and access-fault exceptions.

MISALIGNED_LDST_EXCEPTION_PRIORITY cannot be "high" when MISALIGNED_MAX_ATOMICITY_GRANULE_SIZE is non-zero, since the atomicity of an access cannot be determined in that case until after address translation.

MISALIGNED_MAX_ATOMICITY_GRANULE_SIZE

The maximum granule size, in bytes, that the hart can atomically perform a misaligned load/store/AMO without raising a Misaligned exception. When MISALIGNED_MAX_ATOMICITY_GRANULE_SIZE is 0, the hart cannot atomically perform a misaligned load/store/AMO. When a power of two, the hart can atomically load/store/AMO a misaligned access that is fully contained in a MISALIGNED_MAX_ATOMICITY_GRANULE_SIZE-aligned region.

Even if the hart is capable of performing a misaligned load/store/AMO atomically, a misaligned exception may still occur if the access does not have the appropriate Misaligned Atomicity Granule PMA set.

MISALIGNED_SPLIT_STRATEGY

When misaligned accesses are supported, this determines the **order** in the implementation appears to process the load/store, which determines how/which exceptions will be reported

Options:

- by_byte: The load/store appears to be broken into byte-sized accesses that processed sequentially from smallest address to largest address
- custom: Something else. Will result in a call to unpredictable() in the execution

MISA_CSR_IMPLEMENTED

Whether or not the misa CSR returns zero or a non-zero value.

Possible values:

true

The misa CSR returns a non-zero value.

false

The misa CSR is read-only-0.

MTVAL_WIDTH

The number of implemented bits in the mtval CSR. This is the CSR that may be written when a trap is taken into M-mode with exception-specific information to assist software in handling the trap (e.g., address associated with exception).

Must be greater than or equal to *max*(`PHYS_ADDR_WIDTH`, `VA_SIZE`)

MTVEC_BASE_ALIGNMENT_DIRECT

Byte alignment for mtvec.BASE when mtvec.MODE is Direct.

Cannot be less than 4-byte alignment.

MTVEC_BASE_ALIGNMENT_VECTORED

Byte alignment for mtvec.BASE when mtvec.MODE is Vectored.

Cannot be less than 4-byte alignment.

MTVEC_MODES

Modes supported by mtvec.MODE. If only one, it is assumed to be read-only with that value.

M_MODE_ENDIANESS

Endianness of data in M-mode. Can be one of:

little	M-mode data is always little endian
big	M-mode data is always big endian
dynamic	M-mode data can be either little or big endian, depending on the CSR field mstatus.MBE

PHYS_ADDR_WIDTH

Number of bits in the physical address space.

PMA_GRANULARITY

log2 of the smallest supported PMA region.

Generally, for systems with an MMU, should not be smaller than 12, as that would preclude caching PMP results in the TLB along with virtual memory translations

PRECISE_SYNCHRONOUS_EXCEPTIONS

Whether or not all synchronous exceptions are precise.

If false, any exception not otherwise mandated to precise (e.g., PMP violation) will cause execution to enter an unpredictable state.

REPORT_ENCODING_IN_MTVAL_ON_ILLEGAL_INSTRUCTION

When true, mtval is written with the encoding of an instruction that causes an `IllegalInstruction` exception.

When false mtval is written with `0` when an `IllegalInstruction` exception occurs.

REPORT_VA_IN_MTVAL_ON_BREAKPOINT

When true, mtval is written with the virtual PC of the EBREAK instruction (same information as mepc).

When false, mtval is written with `0` on an EBREAK instruction.

Regardless, mtval is always written with a virtual PC when an external breakpoint is generated

REPORT_VA_IN_MTVAL_ON_INSTRUCTION_ACCESS_FAULT

When true, mtval is written with the virtual PC of an instructino when fetch causes an `InstructionAccessFault`.

WHen false, mtval is written with 0 when an instruction fetch causes an `InstructionAccessFault`.

REPORT_VA_IN_MTVAL_ON_INSTRUCTION_MISALIGNED

When true, mtval is written with the virtual PC when an instruction fetch is misaligned.

When false, mtval is written with 0 when an instruction fetch is misaligned.

Note that when IALIGN=16 (i.e., when the C or one of the `Zc*` extensions are implemented), it is impossible to generate a misaligned fetch, and so this parameter has no effect.

REPORT_VA_IN_MTVAL_ON_LOAD_ACCESS_FAULT

When true, mtval is written with the virtual address of a load when it causes a `LoadAccessFault`.

WHen false, mtval is written with 0 when a load causes a `LoadAccessFault`.

REPORT_VA_IN_MTVAL_ON_LOAD_MISALIGNED

When true, mtval is written with the virtual address of a load instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, mtval is written with 0 when a load address is misaligned and MISALIGNED_LDST is false.

REPORT_VA_IN_MTVAL_ON_STORE_AMO_ACCESS_FAULT

When true, mtval is written with the virtual address of a store when it causes a `StoreAmoAccessFault`.

WHen false, mtval is written with 0 when a store causes a `StoreAmoAccessFault`.

REPORT_VA_IN_MTVAL_ON_STORE_AMO_MISALIGNED

When true, mtval is written with the virtual address of a store instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, mtval is written with 0 when a store address is misaligned and MISALIGNED_LDST is false.

TRAP_ON_EBREAK

Whether or not an EBREAK causes a synchronous exception.

The spec states that implementations may handle EBREAKs transparently without raising a trap, in which case the EEI must provide a builtin.

TRAP_ON_ECALL_FROM_M

Whether or not an ECALL-from-M-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

TRAP_ON_ILLEGAL_WLRL

When true, writing an illegal value to a WLRL CSR field raises an `IllegalInstruction` exception.

When false, writing an illegal value to a WLRL CSR field is `unpredictable`.

TRAP_ON_RESERVED_INSTRUCTION

When true, fetching an unimplemented and/or undefined instruction from the standard/reserved encoding space will cause an `IllegalInstruction` exception.

When false, fetching such an instruction is `UNPREDICTABLE`.

TRAP_ON_UNIMPLEMENTED_CSR

When true, accessing an unimplemented CSR (via a Zicsr instruction) will cause an `IllegalInstruction` exception.

When false, accessing an unimplemented CSR (via a Zicsr instruction) is `unpredictable`.

TRAP_ON_UNIMPLEMENTED_INSTRUCTION

When true, fetching an unimplemented instruction from the custom encoding space will cause an `IllegalInstruction` exception.

When false, fetching an unimplemented instruction is `UNPREDICTABLE`.

VENDOR_ID_BANK

JEDEC Vendor ID bank, for mvendorid

VENDOR_ID_OFFSET

Vendor JEDEC code offset, for mvendorid

XLEN

XLEN in M-mode (AKA MXLEN, tracked by issue #52)

NUM_PMP_ENTRIES

Number of implemented PMP entries. Can be any value between 0-64, inclusive.

The architecture mandates that the number of implemented PMP registers must appear to be 0, 16, or 64.

Therefore, pmp registers will behave as follows according to NUN_PMP_ENTRIES:

NUM_PMP_ENTRIES	pmpaddr<0-15> / pmpcfg<0-3>	pmpaddr<16-63> / pmpcfg<4-15>
0	N	N
1-16	Y	N
17-64	Y	Y

- N = Not implemented; access will cause `IllegalInstruction` if TRAP_ON_UNIMPLEMENTED_CSR is true
- Y = Implemented; access will not cause an exception (from M-mode), but register may be read-only-zero if NUM_PMP_ENTRIES is less than the corresponding register

> `pmpcfgN` *for an odd N never exists when XLEN == 64*

When NUM_PMP_ENTRIES is not exactly 0, 16, or 64, some extant pmp registers, and associated pmpNcfg, will be read-only zero (but will never cause an exception).

PMP_GRANULARITY

log2 of the smallest supported PMP region.

Generally, for systems with an MMU, should not be smaller than 12, as that would preclude caching PMP results in the TLB along with virtual memory translations

Note that PMP_GRANULARITY is equal to G+2 (not G) as described in the privileged architecture.

ASID_WIDTH

Number of implemented ASID bits. Maximum is 16 for XLEN==64, and 9 for XLEN==32

MSTATUS_FS_LEGAL_VALUES

The set of values that mstatus.FS will accept from a software write.

MSTATUS_FS_WRITEABLE

When S is enabled but F is not, mstatus.FS is optionally writeable.

This parameter only has an effect when both S and F mode are disabled.

MSTATUS_TVM_IMPLEMENTED

Whether or not mstatus.TVM is implemented.

When not implemented mstatus.TVM will be read-only-zero.

MSTATUS_VS_LEGAL_VALUES

The set of values that mstatus.VS will accept from a software write.

MSTATUS_VS_WRITEABLE

When S is enabled but V is not, mstatus.VS is optionally writeable.

This parameter only has an effect when both S and V mode are disabled.

MUTABLE_MISA_S

Indicates whether or not the S extension can be disabled with the misa.S bit.

REPORT_ENCODING_IN_STVAL_ON_ILLEGAL_INSTRUCTION

When true, stval is written with the encoding of an instruction that causes an `IllegalInstruction` exception.

When false stval is written with `0` when an `IllegalInstruction` exception occurs.

REPORT_VA_IN_MTVAL_ON_INSTRUCTION_PAGE_FAULT

When true, mtval is written with the virtual PC of an instructino when fetch causes an `InstructionPageFault`.

WHen false, mtval is written with `0` when an instruction fetch causes an `InstructionPageFault`.

REPORT_VA_IN_MTVAL_ON_LOAD_PAGE_FAULT

When true, mtval is written with the virtual address of a load when it causes a `LoadPageFault`.

WHen false, mtval is written with `0` when a load causes a `LoadPageFault`.

REPORT_VA_IN_MTVAL_ON_STORE_AMO_PAGE_FAULT

When true, mtval is written with the virtual address of a store when it causes a `StoreAmoPageFault`.

WHen false, mtval is written with `0` when a store causes a `StoreAmoPageFault`.

REPORT_VA_IN_STVAL_ON_BREAKPOINT

When true, stval is written with the virtual PC of the EBREAK instruction (same information as mepc).

When false, stval is written with `0` on an EBREAK instruction.

Regardless, stval is always written with a virtual PC when an external breakpoint is generated

REPORT_VA_IN_STVAL_ON_INSTRUCTION_ACCESS_FAULT

When true, stval is written with the virtual PC of an instructino when fetch causes an `InstructionAccessFault`.

WHen false, stval is written with `0` when an instruction fetch causes an `InstructionAccessFault`.

REPORT_VA_IN_STVAL_ON_INSTRUCTION_MISALIGNED

When true, stval is written with the virtual PC when an instruction fetch is misaligned.

When false, stval is written with `0` when an instruction fetch is misaligned.

Note that when IALIGN=16 (i.e., when the C or one of the `Zc*` extensions are implemented), it is impossible to generate a misaligned fetch, and so this parameter has no effect.

REPORT_VA_IN_STVAL_ON_INSTRUCTION_PAGE_FAULT

When true, stval is written with the virtual PC of an instructino when fetch causes an `InstructionPageFault`.

WHen false, stval is written with `0` when an instruction fetch causes an `InstructionPageFault`.

REPORT_VA_IN_STVAL_ON_LOAD_ACCESS_FAULT

When true, stval is written with the virtual address of a load when it causes a `LoadAccessFault`.

WHen false, stval is written with `0` when a load causes a `LoadAccessFault`.

REPORT_VA_IN_STVAL_ON_LOAD_MISALIGNED

When true, stval is written with the virtual address of a load instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, stval is written with `0` when a load address is misaligned and MISALIGNED_LDST is false.

REPORT_VA_IN_STVAL_ON_LOAD_PAGE_FAULT

When true, stval is written with the virtual address of a load when it causes a `LoadPageFault`.

WHen false, stval is written with `0` when a load causes a `LoadPageFault`.

REPORT_VA_IN_STVAL_ON_STORE_AMO_ACCESS_FAULT

When true, stval is written with the virtual address of a store when it causes a `StoreAmoAccessFault`.

WHen false, stval is written with 0 when a store causes a `StoreAmoAccessFault`.

REPORT_VA_IN_STVAL_ON_STORE_AMO_MISALIGNED

When true, stval is written with the virtual address of a store instruction when the address is misaligned and MISALIGNED_LDST is false.

When false, stval is written with 0 when a store address is misaligned and MISALIGNED_LDST is false.

REPORT_VA_IN_STVAL_ON_STORE_AMO_PAGE_FAULT

When true, stval is written with the virtual address of a store when it causes a `StoreAmoPageFault`.

WHen false, stval is written with 0 when a store causes a `StoreAmoPageFault`.

SATP_MODE_BARE

Whether or not satp.MODE == Bare is supported.

SCOUNTENABLE_EN

Indicates which counters can delegated via scounteren

An unimplemented counter cannot be specified, i.e., if HPM_COUNTER_EN[3] is false, it would be illegal to set SCOUNTENABLE_EN[3] to true.

SCOUNTENABLE_EN[0:2] must all be false if Zicntr is not implemented. SCOUNTENABLE_EN[3:31] must all be false if Zihpm is not implemented.

STVAL_WIDTH

The number of implemented bits in stval.

Must be greater than or equal to *max*(`PHYS_ADDR_WIDTH`, `VA_SIZE`)

STVEC_MODE_DIRECT

Whether or not stvec.MODE supports Direct (0).

STVEC_MODE_VECTORED

Whether or not stvec.MODE supports Vectored (1).

SV_MODE_BARE

Whether or not writing mode=Bare is supported in the satp register.

SXLEN

Set of XLENs supported in S-mode. Can be one of:

- 32: SXLEN is always 32
- 64: SXLEN is always 64
- 3264: SXLEN can be changed (via mstatus.SXL) between 32 and 64

S_MODE_ENDIANESS

Endianness of data in S-mode. Can be one of:

- little: S-mode data is always little endian

- big: S-mode data is always big endian
- dynamic: S-mode data can be either little or big endian, depending on the CSR field mstatus.SBE

TRAP_ON_ECALL_FROM_S

Whether or not an ECALL-from-S-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY

For implementations that make satp.MODE read-only zero (always Bare, *i.e.*, no virtual translation is implemented), attempts to execute an SFENCE.VMA instruction might raise an illegal-instruction exception.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY indicates whether or not that exception occurs.

TRAP_ON_SFENCE_VMA_WHEN_SATP_MODE_IS_READ_ONLY has no effect when some virtual translation mode is supported.

MUTABLE_MISA_U

Indicates whether or not the U extension can be disabled with the misa.U bit.

TRAP_ON_ECALL_FROM_U

Whether or not an ECALL-from-U-mode causes a synchronous exception.

The spec states that implementations may handle ECALLs transparently without raising a trap, in which case the EEI must provide a builtin.

UXLEN

Set of XLENs supported in U-mode. Can be one of:

- 32: SXLEN is always 32
- 64: SXLEN is always 64
- 3264: SXLEN can be changed (via mstatus.UXL) between 32 and 64

U_MODE_ENDIANESS

Endianness of data in U-mode. Can be one of:

- little: U-mode data is always little endian
- big: U-mode data is always big endian
- dynamic: U-mode data can be either little or big endian, depending on the CSR field mstatus.UBE

LRSC_FAIL_ON_NON_EXACT_LRSC

Whether or not a Store Conditional fails if its physical address and size do not exactly match the physical address and size of the last Load Reserved in program order (independent of whether or not the SC is in the current reservation set)

LRSC_FAIL_ON_VA_SYNONYM

Whether or not an `sc.l`/sc.d will fail if its VA does not match the VA of the prior `lr.l`/lr.d, even if the physical address of the SC and LR are the same

LRSC_MISALIGNED_BEHAVIOR

What to do when an LR/SC address is misaligned and MISALIGNED_AMO == false.

- 'always raise misaligned exception': self-explainitory
- 'always raise access fault': self-explainitory
- 'custom': Custom behavior; misaligned LR/SC may sometimes raise a misaligned exception and sometimes raise a access fault. Will lead to an 'unpredictable' call on any misaligned LR/SC access

LRSC_RESERVATION_STRATEGY

Strategy used to handle reservation sets.

- "reserve naturally-aligned 64-byte region": Always reserve the 64-byte block containing the LR/SC address
- "reserve naturally-aligned 128-byte region": Always reserve the 128-byte block containing the LR/SC address
- "reserve exactly enough to cover the access": Always reserve exactly the LR/SC access, and no more
- "custom": Custom behavior, leading to an 'unpredictable' call on any LR/SC

MISALIGNED_AMO

whether or not the implementation supports misaligned atomics in main memory

MUTABLE_MISA_A

When the A extensions is supported, indicates whether or not the extension can be disabled in the misa.A bit.

MUTABLE_MISA_B

Indicates whether or not the B extension can be disabled with the misa.B bit.

MUTABLE_MISA_M

Indicates whether or not the M extension can be disabled with the misa.M bit.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

FORCE_UPGRADE_CBO_INVAL_TO_FLUSH

When true, an implementation prohibits setting menvcfg.CBIE == 11 such that all cbo.inval instructions either trap (when menvcfg.CBIE == '00') or flush (when menvcfg.CBIE == '01').

When false, an implementation allows a true INVAL operation for cbo.inval, and thus supports the setting menvcfg.CBIE == 11.

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

CACHE_BLOCK_SIZE

The observable size of a cache block, in bytes

TIME_CSR_IMPLEMENTED

Whether or not a real hardware time CSR exists. Implementations can either provide a real CSR or emulate access at M-mode.

Possible values:

true

time/timeh exists, and accessing it will not cause an IllegalInstruction trap

false

time/timeh does not exist. Accessing the CSR will cause an IllegalInstruction trap or enter an unpredictable state, depending on TRAP_ON_UNIMPLEMENTED_CSR. Privileged software may emulate the time CSR, or may pass the exception to a lower level.

HW_MSTATUS_FS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.FS

Values are:

never	Hardware never writes mstatus.FS
precise	Hardware writes mstatus.FS to the Dirty (3) state precisely when F registers are modified
imprecise	Hardware writes mstatus.FS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write FP state.

MSTATUS_FS_LEGAL_VALUES

The set of values that mstatus.FS will accept from a software write.

MUTABLE_MISA_F

Indicates whether or not the F extension can be disabled with the misa.F bit.

MUTABLE_MISA_D

Indicates whether or not the D extension can be disabled with the misa.D bit.

HW_MSTATUS_VS_DIRTY_UPDATE

Indicates whether or not hardware will write to mstatus.VS

Values are:

never	Hardware never writes mstatus.VS
precise	Hardware writes mstatus.VS to the Dirty (3) state precisely when V registers are modified
imprecise	Hardware writes mstatus.VS imprecisely. This will result in a call to unpredictable() on any attempt to read mstatus or write vector state.

MSTATUS_VS_LEGAL_VALUES

The set of values that mstatus.VS will accept from a software write.

MUTABLE_MISA_V

Indicates whether or not the V extension can be disabled with the misa.V bit.

Appendix A: Profile Comparisons

A.1. Microcontroller Profile Releases

The Microcontroller processor kind has 1 processor profile releases that reference a total of 51 extensions.

Table 1. Extension Presence

Extension	RT-Europa
A	mandatory
B	mandatory
D	optional
F	optional
I	mandatory
M	mandatory
S	mandatory
Sha	optional
Sm	mandatory
Smaia	mandatory
Smcdeleg	optional
Smclic	mandatory
Smcntrpmf	optional
Smdbltrp	optional
Smepmp	mandatory
Smpmp	mandatory
Smrnmi	mandatory
Ssaia	mandatory
Ssccfg	optional
Ssclic	mandatory
Sscofpmf	optional
Sscounterenw	optional
Sspmp	optional
Ssstrict	mandatory
Sstc	mandatory
Sstvecd	mandatory
Svbare	optional
U	mandatory
V	optional

Extension	RT-Europa
Zce	mandatory
Zcf	optional
Zclsd	optional
Zcmop	mandatory
Zfa	optional
Zfbfmin	optional
Zfh	optional
Zicbom	mandatory
Zicbop	mandatory
Zicboz	mandatory
Ziccamoa	mandatory
Zicntr	mandatory
Zicond	mandatory
Zicsr	mandatory
Zifencei	mandatory
Zihintntl	mandatory
Zihintpause	mandatory
Zihpm	mandatory
Zilsd	optional
Zimop	mandatory
Zvfbfmin	optional
Zvfbfwma	optional

A.2. QNT-RT Profile Releases

The QNT-RT Profile Class has 1 releases that reference a total of 51 extensions.

Table 2. Extension Presence

Extension	RT-Europa
A	mandatory
B	mandatory
D	optional
F	optional
I	mandatory
M	mandatory
S	mandatory

Extension	RT-Europa
Sha	optional
Sm	mandatory
Smaia	mandatory
Smcdeleg	optional
Smclic	mandatory
Smcntrpmf	optional
Smdbltrp	optional
Smepmp	mandatory
Smpmp	mandatory
Smrnmi	mandatory
Ssaia	mandatory
Ssccfg	optional
Ssclic	mandatory
Sscofpmf	optional
Sscounterenw	optional
Sspmp	optional
Ssstrict	mandatory
Sstc	mandatory
Sstvecd	mandatory
Svbare	optional
U	mandatory
V	optional
Zce	mandatory
Zcf	optional
Zclsd	optional
Zcmop	mandatory
Zfa	optional
Zfbfmin	optional
Zfh	optional
Zicbom	mandatory
Zicbop	mandatory
Zicboz	mandatory
Ziccamoa	mandatory
Zicntr	mandatory
Zicond	mandatory

Extension	RT-Europa
Zicsr	mandatory
Zifencei	mandatory
Zihintntl	mandatory
Zihintpause	mandatory
Zihpm	mandatory
Zilsd	optional
Zimop	mandatory
Zvfbfmin	optional
Zvfbfwma	optional

A.3. RT-Europa Profiles

The RT-Europa Profile Release has 3 profiles that reference a total of 51 extensions.

Extensions present in a profile are also present in higher-privileged profiles in the same profile release.

Table 3. Extension Presence

Extension	RT-EuropaU32	RT-EuropaS32	RT-EuropaM32
A	mandatory (2.1.0)	mandatory (2.1.0)	mandatory (2.1.0)
B	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
D	optional (2.2.0)	optional (2.2.0)	optional (2.2.0)
F	optional (2.2.0)	optional (2.2.0)	optional (2.2.0)
I	mandatory (2.1.0)	mandatory (2.1.0)	mandatory (2.1.0)
M	mandatory (2.0.0)	mandatory (2.0.0)	mandatory (2.0.0)
S	- (-)	mandatory (1.13.0)	mandatory (1.13.0)
Sha	- (-)	optional (1.0.0)	optional (1.0.0)
Sm	- (-)	- (-)	mandatory (1.13.0)
Smaia	- (-)	- (-)	mandatory (1.0.0)
Smcdeleg	- (-)	- (-)	optional (1.0.0)
Smclic	- (-)	- (-)	mandatory (0.9.0)
Smcntrpmf	- (-)	- (-)	optional (1.0.0)
Smdbltrp	- (-)	- (-)	optional (1.0.0)
Smepmp	- (-)	- (-)	mandatory (1.0.0)
Smpmp	- (-)	- (-)	mandatory (1.13.0)
Smrnmi	- (-)	- (-)	mandatory (1.0.0)
Ssaia	- (-)	mandatory (1.0.0)	mandatory (1.0.0)

Extension	RT-EuropaU32	RT-EuropaS32	RT-EuropaM32
Ssccfg	- (-)	optional (1.0.0)	optional (1.0.0)
Ssclic	- (-)	mandatory (0.9.0)	mandatory (0.9.0)
Sscofpmf	- (-)	optional (1.0.0)	optional (1.0.0)
Sscounterenw	- (-)	optional (1.0.0)	optional (1.0.0)
Sspmp	- (-)	optional (0.9.2)	optional (0.9.2)
Ssstrict	- (-)	mandatory (1.0.0)	mandatory (1.0.0)
Sstc	- (-)	mandatory (0.9.0)	mandatory (0.9.0)
Sstvecd	- (-)	mandatory (1.0.0)	mandatory (1.0.0)
Svbare	- (-)	optional (1.0.0)	optional (1.0.0)
U	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
V	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zce	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zcf	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zclsd	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zcmop	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zfa	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zfbfmin	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zfh	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zicbom	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zicbop	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zicboz	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Ziccamoa	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zicntr	mandatory (2.0.0)	mandatory (2.0.0)	mandatory (2.0.0)
Zicond	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zicsr	mandatory (2.0.0)	mandatory (2.0.0)	mandatory (2.0.0)
Zifencei	- (-)	mandatory (2.0.0)	mandatory (2.0.0)
Zihintntl	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zihintpause	mandatory (2.0.0)	mandatory (2.0.0)	mandatory (2.0.0)
Zihpm	mandatory (2.0.0)	mandatory (2.0.0)	mandatory (2.0.0)
Zilsd	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zimop	mandatory (1.0.0)	mandatory (1.0.0)	mandatory (1.0.0)
Zvfbfmin	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)
Zvfbfwma	optional (1.0.0)	optional (1.0.0)	optional (1.0.0)

QUINTAURIS